Kimi TypeMaster - Complete Project Overview

📋 Table of Contents

1. Project Description
2. Technology Stack
3. Architecture Overview
4. Project Structure
5. Core Features
6. System Components
7. Database Models
8. API Architecture
9. Real-Time Features
10. Setup & Installation
11. Development Workflow

Project Description

Kimi TypeMaster is a comprehensive multiplayer typing competition platform built with modern web technologies. The application enables users to compete in real-time typing races, participate in daily challenges, climb ranked ladders, and form squads for team competitions.

Purpose

- **Racing Competitions**: Real-time multiplayer typing races
- **Skill Development**: Daily challenges with progressive difficulty

- **Competitive Rankings**: Ranked ladder system with tier progression
- **Social Features**: Squads, invitations, and friend leaderboards
- **Tournament Support**: Organized competitive events
- **Admin Management**: Comprehensive admin dashboard for system management

Target Users

- Casual typists wanting to improve their speed
- Competitive players seeking ranked competitions
- Users interested in tournament play
- Groups/squads competing together

Technology Stack

Frontend

Technology	Purpose
React 19.2.0	UI framework
TypeScript	Type safety
Vite	Build tool and dev server
Socket.io-client	Real-time communication
Tailwind CSS	Styling
Radix UI	Component library
React Hook Form	Form management
Lucide React	Icons

Backend

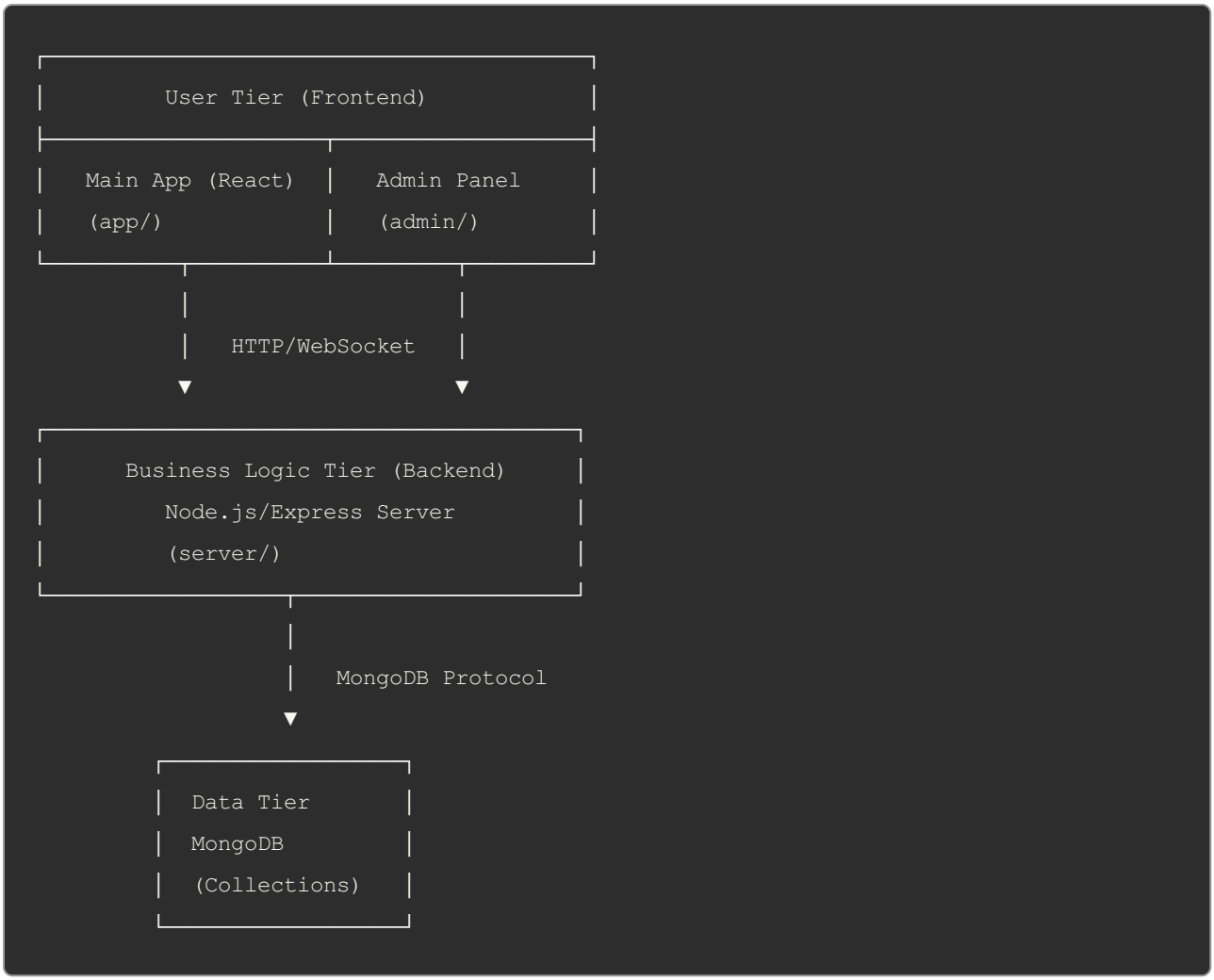
Technology	Purpose
Node.js	Runtime environment
Express.js	Web framework
MongoDB	NoSQL database
Mongoose	ODM for MongoDB
Socket.io	Real-time events
JWT	Authentication
bcryptjs	Password hashing
CORS	Cross-origin requests

Build & Development

Tool	Purpose
npm	Package manager
ESLint	Code linting
PostCSS	CSS processing
Nodemon	Backend auto-reload

Architecture Overview

The application follows a **three-tier architecture**:



Component Breakdown

Frontend (app/)

- User-facing typing competition interface
- Real-time race display
- User profiles and leaderboards
- Race creation and joining
- Daily challenge participation

Admin Panel (admin/)

- Independent React application
- System administration tools
- User management
- Event scheduling

- Analytics and reporting

Backend (server/)

- Express API server
- Socket.io real-time server
- Business logic implementation
- Database interactions
- Authentication and authorization

Project Structure

```
Kimi_Agent_Typing App Creation/
│
├── 📄 Documentation Files
│   ├── PROJECT_OVERVIEW.md (this file)
│   ├── SETUP_GUIDE.md
│   ├── IMPLEMENTATION_SUMMARY.md
│   ├── DAILY_CHALLENGE_GUIDE.md
│   ├── RANKING_SYSTEM_GUIDE.md
│   ├── FALLING_WORDS_IMPLEMENTATION.md
│   └── SQUAD_RATING_VERIFICATION.md
│
├── 📁 app/ (Main Frontend Application)
│   ├── src/
│   │   ├── components/         # React components
│   │   ├── pages/              # Page components
│   │   ├── hooks/              # Custom hooks
│   │   ├── lib/
│   │   │   ├── api.ts          # API client
│   │   │   ├── socket.ts       # Socket.io client
│   │   │   ├── utils.ts        # Utility functions
│   │   │   └── notifications.ts # Notification system
│   │   ├── types/              # TypeScript type definitions
│   │   ├── App.tsx             # Main app component
│   │   ├── main.tsx            # Entry point
│   │   └── index.css           # Global styles
│   ├── public/                 # Static assets
│   ├── vite.config.ts          # Vite configuration
```

```
│   ├── tailwind.config.js      # Tailwind CSS config
│   ├── tsconfig.json           # TypeScript config
│   ├── package.json            # Dependencies
│   └── index.html              # HTML entry point
│
├── 🗀 admin/ (Admin Panel Application)
│   ├── src/
│   │   ├── components/         # Admin UI components
│   │   │   ├── AdminPage.tsx
│   │   │   ├── AuditDashboard.tsx
│   │   │   ├── NotificationContainer.tsx
│   │   │   └── ui/             # Reusable UI components
│   │   ├── pages/              # Admin page views
│   │   │   ├── AnalyticsPage.tsx
│   │   │   ├── DailyChallengesPage.tsx
│   │   │   ├── DashboardPage.tsx
│   │   │   ├── LeaderboardsPage.tsx
│   │   │   └── LivePlayersPage.tsx
│   │   ├── lib/
│   │   │   ├── api.ts          # Admin API client
│   │   │   ├── socket.ts       # Socket connection
│   │   │   ├── notifications.ts
│   │   │   └── utils.ts
│   │   ├── AdminApp.tsx
│   │   ├── admin-main.tsx      # Entry point
│   │   └── index.css
│   ├── public/                 # Static assets
│   ├── vite.config.ts
│   ├── tailwind.config.js
│   ├── tsconfig.json
│   ├── package.json
│   └── index.html
│
└── 🗀 server/ (Backend Application)
    ├── src/ OR root/
    │   ├── server.js           # Express app setup
    │   ├── models/             # Mongoose schemas
    │   │   ├── User.js
    │   │   ├── Race.js
    │   │   ├── RaceResult.js
    │   │   ├── DailyChallenge.js
    │   │   ├── UserRanking.js
    │   │   ├── Leaderboard.js
    │   │   ├── Squad.js
```

```
│   │   ├── SquadInvite.js
│   │   ├── Tournament.js
│   │   ├── Invitation.js
│   │   ├── Payment.js
│   │   ├── PrizePool.js
│   │   ├── Activity.js
│   │   ├── Report.js
│   │   ├── Settings.js
│   │   ├── CompetitionRegistration.js
│   │   ├── SquadWar.js
│   │   └── others
│   ├── routes/             # API endpoints
│   │   ├── auth.js
│   │   ├── race.js
│   │   ├── user.js
│   │   ├── leaderboard.js
│   │   ├── dailyChallenge.js
│   │   ├── squad.js
│   │   ├── tournament.js
│   │   ├── payment.js
│   │   ├── invitation.js
│   │   ├── admin.js
│   │   ├── livePlayer.js
│   │   └── others
│   ├── lib/                # Business logic utilities
│   │   ├── rankingSystem.js
│   │   ├── dailyChallengeUtils.js
│   │   ├── tournamentUtils.js
│   │   ├── squadUtils.js
│   │   ├── challengeScheduler.js
│   │   ├── competitionUtils.js
│   │   ├── paymentUtils.js
│   │   ├── idGenerator.js
│   │   ├── wordList.js
│   │   └── others
│   ├── socket/             # Real-time event handlers
│   │   ├── raceHandler.js
│   │   └── livePlayersHandler.js
│   ├── middleware/
│   │   └── auth.js         # JWT verification
│   ├── config/             # Configuration files
│   ├── package.json
│   ├── .env                # Environment variables
│   └── server utilities
```

```
│
└── 📄 Root Configuration Files
    └── package.json            # Workspace root
```

Core Features

1. Multiplayer Typing Races ⌨️

- Real-time competitive typing races
- 2-8 player rooms
- Custom race text or predefined passages
- Difficulty levels (beginner, intermediate, advanced)
- Live leaderboard updates
- Instant result calculation
- Shareable invite codes for races

Key Files:

- server/models/Race.js (server/models/Race.js)
- server/routes/race.js (server/routes/race.js)
- server/socket/raceHandler.js (server/socket/raceHandler.js)

2. Daily Challenges 🧮

- Time-limited typing competitions (24-hour cycles)
- Same challenge text for all players
- Automatic leaderboard ranking
- Rating point rewards (top 10 placements)
- Performance tracking across multiple days
- Progressive difficulty increase
- Customizable challenge content

Key Features:

- Score calculation: `(WPM/targetWPM × 50) + Accuracy`
- Reward system: 500 points for 1st place, decreasing to 25 for 10th
- Challenge reset at midnight UTC
- Historical challenge archives

Key Files:

- server/models/DailyChallenge.js (server/models/DailyChallenge.js)
- server/lib/dailyChallengeUtils.js (server/lib/dailyChallengeUtils.js)
- server/routes/dailyChallenge.js (server/routes/dailyChallenge.js)

3. Ranked Ladder System 🎯

- Elo-based rating point system
- 5 competitive tiers: Bronze → Silver → Gold → Platinum → Diamond
- Win/loss tracking with streak system
- Average WPM and accuracy statistics
- Tier-specific leaderboards
- Seasonal rating resets
- Historical rating tracking

Tier Structure:

```
Bronze:     0 - 1499     🥉
Silver:     1500 - 2499  🥈
Gold:       2500 - 3499  🥇
Platinum:   3500 - 4999  💎
Diamond:    5000+        👑
```

Key Files:

- server/models/UserRanking.js (server/models/UserRanking.js)
- server/lib/rankingSystem.js (server/lib/rankingSystem.js)
- server/routes/leaderboard.js (server/routes/leaderboard.js)

4. Squads & Team Competitions 👥

- Create and join squads (teams)
- Squad rankings and leaderboards
- Squad wars (team vs team competitions)
- Role-based permissions (leader, member)
- Squad invitations
- Collective statistics

Key Files:

- server/models/Squad.js (server/models/Squad.js)
- server/models/SquadWar.js (server/models/SquadWar.js)
- server/routes/squad.js (server/routes/squad.js)

5. Tournament System 🏆

- Create and manage tournaments
- Bracket-based or round-robin formats
- Prize pool distribution
- Team and individual tournaments
- Registration management
- Real-time tournament progress tracking

Key Files:

- server/models/Tournament.js (server/models/Tournament.js)
- server/lib/tournamentUtils.js (server/lib/tournamentUtils.js)
- server/routes/tournament.js (server/routes/tournament.js)

6. Social Features 💬

- Friend management
- Friend invitations
- Private messaging capability
- User search functionality
- Profile customization

- Friends leaderboards

Key Files:

- server/models/Invitation.js (server/models/Invitation.js)
- server/routes/invitation.js (server/routes/invitation.js)
- server/routes/user.js (server/routes/user.js)

7. Payment & Monetization 💳

- Prize pool creation and management
- Payment processing
- Transaction history
- Reward distribution
- Bonus point system

Key Files:

- server/models/Payment.js (server/models/Payment.js)
- server/models/PrizePool.js (server/models/PrizePool.js)
- server/lib/paymentUtils.js (server/lib/paymentUtils.js)

8. Admin Dashboard 🎛️

- User management
- Event scheduling
- Analytics and reporting
- Live player monitoring
- Leaderboard verification
- Audit logging
- System settings

Key Files:

- admin/src/components/AdminPage.tsx (admin/src/components/AdminPage.tsx)
- admin/src/pages/ (admin/src/pages/)

9. Live Features

- Real-time race updates via WebSocket
- Live player tracking
- Instant leaderboard updates
- Concurrent user monitoring
- Activity feeds

System Components

Frontend Architecture

Components Layer

```
App.tsx
├── Pages (Route-based)
│   ├── RaceHub - Race creation & joining
│   ├── LiveRace - Active race display
│   ├── Profile - User profile
│   ├── Leaderboard - Rankings display
│   └── DailyChallenge - Daily challenge UI
└── UI Components (Radix + Custom)
    ├── Buttons, Forms, Dialogs
    ├── Modals, Cards, Lists
    └── Custom Widgets (WPM display, etc)
```

API Client Layer

lib/api.ts - Centralized API management

```
- auth: register(), login(), verify()
- races: create(), join(), list(), getDetails()
- users: getProfile(), updateProfile(), search()
- leaderboard: getGlobal(), getTier(), getUserRank()
- dailyChallenge: submit(), getLeaderboard(), getHistory()
- squads: create(), join(), invite()
- tournaments: register(), getList()
```

Socket.io Layer

lib/socket.ts - Real-time event management

```
Events:
- race:join - Player joins race
- race:ready - Player marks ready
- race:countdown - Countdown sequence
- race:start - Race begins
- race:input - WPM/accuracy updates
- race:finish - Player completes
- race:results - Final leaderboard
- leaderboard:update - Ranking updates
```

Backend Architecture

API Tier (Routes)

Each route file handles specific domain:

- **auth.js** - Authentication endpoints
- **race.js** - Race management
- **user.js** - User profile operations
- **leaderboard.js** - Ranking queries
- **dailyChallenge.js** - Daily challenge operations
- **squad.js** - Squad operations
- **tournament.js** - Tournament management
- **payment.js** - Payment handling

- **admin.js** - Admin operations

Business Logic Tier (lib/)

Utility functions for complex operations:

- **rankingSystem.js** - Elo calculations, tier updates
- **dailyChallengeUtils.js** - Challenge scheduling, scoring
- **tournamentUtils.js** - Bracket generation, match logic
- **squadUtils.js** - Squad management logic
- **challengeScheduler.js** - Automated challenge creation
- **paymentUtils.js** - Payment processing

Real-Time Tier (socket/)

WebSocket event handlers:

- **raceHandler.js** - Race event management
- **livePlayersHandler.js** - Live user tracking

Data Tier (models/)

Mongoose schemas with:

- Data validation
- Relationships and references
- Indexes for performance
- Custom methods

Database Models

Core Models

Model	Purpose	Key Fields
User	User accounts	username, email, passwordHash, stats, tier
Race	Race rooms	participants, text, difficulty, status
RaceResult	Race scores	userId, raceId, wpm, accuracy, rank
DailyChallenge	Daily competitions	date, text, submissions[], leaderboard
UserRanking	Competitive rating	ratingPoints, tier, winRate, streak
Leaderboard	Cached rankings	timeframe, entries[], lastUpdated
Squad	Team groups	leader, members, stats
Tournament	Tournaments	brackets, participants, prizes
Payment	Transactions	userId, amount, status, type
Invitation	Invites	from, to, type (race/friend), status

Relationships

```
User ──→ UserRanking (1:1)
  ├──→ Race (M:N through RaceResult)
  ├──→ DailyChallenge (M:N through submission)
  ├──→ Squad (M:N as member)
  ├──→ Tournament (M:N as participant)
  ├──→ Invitation (receives/sends)
  └──→ Payment (transactions)

Race ──→ RaceResult (1:M)
       ──→ Participant (M:M)

Squad ──→ SquadWar (1:M)
       ──→ Member (M:M)

Tournament ──→ CompetitionRegistration (1:M)
```

API Architecture

Authentication

```
POST /api/auth/register
  Body: { username, email, password }
  Response: { user, token }

POST /api/auth/login
  Body: { email, password }
  Response: { user, token }

GET /api/auth/verify
  Headers: { Authorization: Bearer {token} }
  Response: { valid, user }
```

Races

```
POST /api/races/create
  Body: { text, difficulty, maxPlayers }
  Response: { race, inviteCode }

POST /api/races/:roomId/join
  Body: { code? }
  Response: { race, participants }

GET /api/races
  Response: { races[] }

GET /api/races/:roomId
  Response: { race }
```

User Profile

```
GET /api/users/me
  Response: { user, stats, ranking }

GET /api/users/:userId
  Response: { user, stats, achievements }

PUT /api/users/:userId
  Body: { ...updateData }
  Response: { user }
```

Leaderboard

```
GET /api/leaderboard/global
  Query: { page, limit }
  Response: { leaderboard[], total, pages }

GET /api/leaderboard/tier/:tier
  Response: { leaderboard[] }

GET /api/leaderboard/me
  Response: { rank, tier, stats }
```

Daily Challenge

```
GET /api/dailyChallenge
  Response: { challenge }

POST /api/dailyChallenge/submit
  Body: { wpm, accuracy, time }
  Response: { score, rank, reward }

GET /api/dailyChallenge/leaderboard
  Response: { leaderboard[] }
```

Squads

```
POST /api/squads/create
  Body: { name, description }
  Response: { squad }

POST /api/squads/:squadId/join
  Body: { code }
  Response: { squad }

POST /api/squads/:squadId/invite
  Body: { userId }
  Response: { invitation }
```

Real-Time Features

WebSocket Events

Race Events

```
EMIT (Client → Server):
- race:ready - Player ready for race
- race:input - Send typing progress (WPM, accuracy)
- race:finish - Player completed race

LISTEN (Server → Client):
- race:join - New player joined
- race:update - Race state update
- race:countdown - Race starting countdown (3-2-1)
- race:start - Race has started
- race:input - Broadcast of players' progress
- race:finish - Player finished
- race:results - Final results
- race:error - Error message
```

Live Player Events

```
LISTEN:
- players:online - Active user count
- players:update - User status changes
- activity:log - Live activity feed
```

Connection Management

- Automatic reconnection on disconnect
- Heartbeat/ping-pong to maintain connection
- Connection pooling for multiple rooms
- Event queuing during disconnection

Setup & Installation

Prerequisites

- **Node.js** 16+ (https://nodejs.org (https://nodejs.org))
- **MongoDB** 4.0+ (https://www.mongodb.com/try/download/community (https://www.mongodb.com/try/download/community))

- **npm** (included with Node.js)
- **Git** (for version control)

Step-by-Step Installation

1. MongoDB Setup

Windows:

```
# Download and install MongoDB Community from:
# https://www.mongodb.com/try/download/community

# Or run mongod manually
"C:\Program Files\MongoDB\Server\7.0\bin\mongod.exe"
```

macOS:

```
brew tap mongodb/brew
brew install mongodb-community
brew services start mongodb-community
```

Linux:

```
sudo apt-get install -y mongodb-org
sudo systemctl start mongod
```

2. Backend Setup

```
# Navigate to server directory
cd server

# Install dependencies
npm install

# Create .env file
echo "MONGO_URI=mongodb://localhost:27017/typemaster
JWT_SECRET=your_super_secret_jwt_key_change_in_production
PORT=5000
NODE_ENV=development
CLIENT_URI=http://localhost:5173" > .env

# Start development server
npm run dev
# Output: ☑ MongoDB connected
#         🚀 Server running on port 5000
```

3. Frontend Setup

```
# In new terminal, navigate to app directory
cd app

# Install dependencies
npm install

# Start development server
npm run dev
# Output: VITE v... ready in ... ms
#         ➜  Local:   http://localhost:5173/
```

4. Admin Panel Setup

```
# In new terminal, navigate to admin directory
cd admin

# Install dependencies
npm install

# Create .env file
echo "VITE_API_URL=http://localhost:5000/api" > .env

# Start development server
npm run dev
# Output: VITE v... ready in ... ms
#         ➜  Local:   http://localhost:3000/
```

Accessing the Application

Service	URL	Purpose
Main App	http://localhost:5173	User interface
Admin Panel	http://localhost:3000	Administration
Backend API	http://localhost:5000	API endpoints
WebSocket	ws://localhost:5000	Real-time events
MongoDB	mongodb://localhost:27017	Database

Development Workflow

Project Development Commands

Backend Development

```
cd server

# Start development server (auto-restart on changes)
npm run dev

# Create admin user
npm run create-admin

# Run tests (if available)
npm test
```

Frontend Development

```
cd app

# Start dev server (hot reload)
npm run dev

# Build for production
npm run build

# Preview production build
npm run preview

# Run linting
npm run lint
```

Admin Panel Development

```
cd admin

# Start dev server
npm run dev

# Build for production
npm run build

# Lint code
npm run lint
```

Development Best Practices

Code Organization

- Components in `components/` folder
- Pages in `pages/` folder
- Business logic in `lib/` utilities
- Types in `types/` directory
- Keep components focused and single-responsibility

API Communication

- Use centralized API client (`lib/api.ts`)
- Implement proper error handling
- Add loading states
- Use TypeScript types
- Log API calls in development

Real-Time Features

- Connect to socket on app load
- Handle disconnection gracefully
- Implement reconnection logic
- Clean up listeners on component unmount
- Test with network throttling

Database Queries

- Use proper indexing for frequently queried fields
- Implement pagination for large datasets
- Cache where appropriate
- Monitor query performance
- Use lean() for read-only queries

File Modification Guide

Adding a New Feature

1. Create database model in `server/models/`
2. Create API routes in `server/routes/`
3. Add utility functions in `server/lib/`
4. Create React components in `app/src/components/`
5. Add API client methods in `app/src/lib/api.ts`
6. Update Socket.io handlers if needed
7. Add TypeScript types in `app/src/types/`
8. Test thoroughly

Debugging

Backend Debugging:

- Check server console for errors
- Use MongoDB Compass to inspect data
- Add console.log statements
- Monitor WebSocket connections
- Check browser DevTools Network tab

Frontend Debugging:

- Use React DevTools extension
- Check browser console
- Use Vite DevTools
- Monitor Network tab for API calls
- Test Socket.io connection in console

Performance Optimization

Frontend

- Code splitting with lazy loading
- Image optimization
- Memoization of expensive components
- Virtual scrolling for large lists
- Debounce/throttle user inputs

Backend

- Database indexing
- Query optimization
- Caching strategies
- Connection pooling
- Rate limiting

Real-Time

- Batch WebSocket updates
- Optimize payload sizes
- Implement backpressure
- Monitor connection stability

Additional Resources

Documentation Files

- SETUP_GUIDE.md (SETUP_GUIDE.md) - Quick start guide
- DAILY_CHALLENGE_GUIDE.md (DAILY_CHALLENGE_GUIDE.md) - Daily challenge details
- RANKING_SYSTEM_GUIDE.md (RANKING_SYSTEM_GUIDE.md) - Ranking system documentation
- FALLING_WORDS_IMPLEMENTATION.md (FALLING_WORDS_IMPLEMENTATION.md) - Special feature guide
- SQUAD_RATING_VERIFICATION.md (SQUAD_RATING_VERIFICATION.md) - Squad

rating details

External Resources

- Express.js Documentation (https://expressjs.com)
- React Documentation (https://react.dev)
- MongoDB Documentation (https://docs.mongodb.com)
- Socket.io Documentation (https://socket.io/docs)
- TypeScript Documentation (https://www.typescriptlang.org/docs)

Project Statistics

Metric	Value
Backend Models	13+
API Routes	15+
Frontend Components	20+
Admin Pages	6+
Real-time Events	10+
UI Components (Radix)	30+

Contributors & Maintenance

This project is actively maintained with regular updates to features, security patches, and performance improvements. For contributions or issues, please refer to the project repository.

Last Updated: May 2026
Version: 1.0.0
Status: Active Development